PHASE 5 OF 18

Methods, Classes & Objects

OOP vs procedural, classes and objects in memory, garbage collection basics, variable types, methods, pass-by-value, varargs, recursion, constructors, this, static and initialization order

IN THIS PHASE

5.1 Procedural vs Object-Oriented Programming
5.2 Classes and Objects
5.3 Objects in Memory
5.4 Garbage Collection Basics
5.5 Variables in Depth: Instance, Static and Local
5.6 Methods
5.7 Java Is Always Pass-by-Value
5.8 Variable Arguments (Varargs)
5.9 Recursion
5.10 Constructors
5.11 The this Keyword
5.12 The static Keyword
5.13 Initialization Blocks and Execution Order
5.14 Predict the Output
5.15 Interview Questions
5.16 Quick Revision

5.1 Procedural vs Object-Oriented Programming

Aspect	Procedural (POP)	Object-Oriented (OOP)
Program is built from	Functions / procedures	Classes and objects
Focus	Functions (steps)	Data + behaviour together
Data	Often global, freely accessible	Hidden inside objects (`private`), accessed through methods
Security	Low	High: data hiding and access control
Reuse	Copy or call functions	Inheritance, composition, polymorphism
Maintenance of large apps	Harder	Easier: modular, changes stay local
Examples	C, COBOL, Pascal	Java, C++, C#, Python, Kotlin

Simula 67 is considered the first object-oriented language. OOP rests on four pillars, which Phases 6 and 7 cover in depth:

Pillar	One-line meaning	Achieved with
Encapsulation	Bundle data and methods together and protect the data	`private` fields + getters/setters
Abstraction	Show *what* an object does, hide *how*	Abstract classes, interfaces
Inheritance	Reuse and extend an existing class (IS-A)	`extends`, `implements`
Polymorphism	One name, many forms	Overloading (compile time), overriding (runtime)

5.2 Classes and Objects

A **class** is a blueprint or template: it declares the **state** (fields), **behaviour** (methods) and **construction** (constructors) of a type. An **object** is a concrete **instance** of a class that exists in memory at runtime.

```
public class Student {                  // class: the blueprint
    int id;                             // fields = state
    String name;

    Student(int id, String name) {      // constructor = how to build one
        this.id = id;
        this.name = name;
    }

    void display() {                    // method = behaviour
        System.out.println(id + " " + name);
    }
}

Student s1 = new Student(101, "Raju");   // object 1
Student s2 = new Student(102, "Ravi");   // object 2: own copy of id and name
s1.display();                            // 101 Raju
```

```
Student s1 = new Student(101, "Raju");
  |       |    |     |
  |       |    |     +-- constructor call: initialises the new object
  |       |    +-------- new: allocates memory on the heap, returns its reference
  |       +------------- reference variable (on the stack if local)
  +--------------------- type of the reference (the class)
```

Class	Object
Blueprint / template / user-defined type	Instance of a class
Logical entity, declared once	Physical entity, created many times
Takes no heap memory for its fields (metadata lives in Metaspace)	Occupies heap memory for its own fields
Declared with the `class` keyword	Created with `new` (mostly)
Example: `Student`	Example: `s1`, `s2`

Ways to create an object

Way	Example
`new` keyword (most common)	`Student s = new Student();`
Static factory method	`List.of(1, 2)`, `LocalDate.now()`, `Integer.valueOf(5)`
`clone()`	`Student copy = (Student) s.clone();` (Phase 8)
Reflection	`Student.class.getDeclaredConstructor().newInstance();`
Deserialization	`(Student) objectInputStream.readObject();` (Phase 12)

References, null and anonymous objects

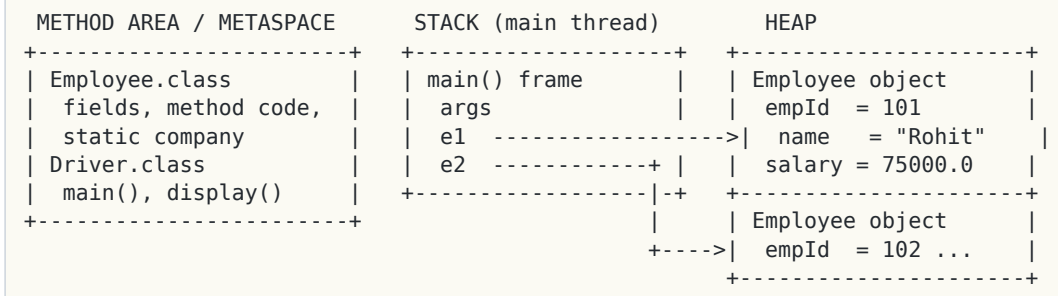
```
Student a = new Student(1, "A");
Student b = a;               // copies the REFERENCE: a and b point to the same object
b.name = "Changed";
System.out.println(a.name);  // Changed

Student c = null;            // refers to no object
c.display();                 // NullPointerException

new Student(5, "Temp").display();   // anonymous object: used once, then GC-eligible
```

5.3 Objects in Memory

```
public class Driver {
    public static void main(String[] args) {
        Employee e1 = new Employee(101, "Rohit", 75000);
        Employee e2 = new Employee(102, "Amit", 65000);
    }
}
```



What happens when a method returns an object

```
class Person { int id; String name; int age; }

class Driver {
    Person getPerson() {
        Person p = new Person();   // object on the heap, defaults: 0, null, 0
        p.id = 101; p.name = "Raju"; p.age = 20;
        return p;                  // returns the REFERENCE; the object is not copied
    }
    public static void main(String[] args) {
        Driver d = new Driver();
        Person p = d.getPerson();
        System.out.println(p.id + " " + p.name + " " + p.age);   // 101 Raju 20
    }
}
```

1. `javac` compiles both classes; the JVM loads `Driver.class` and `Person.class` into the method area.
2. The JVM calls `main()`, which pushes a **stack frame** for `main`.
3. `new Driver()` creates a Driver object on the heap; local `d` (stack) holds its reference.
4. Calling `d.getPerson()` pushes a new frame. `new Person()` allocates the object and fields get **default values**, then the explicit assignments run.
5. `return p` copies the **reference** back to `main`. The `getPerson` frame is popped, but the object survives because `main` still references it.
6. When `main` ends, its frame is popped. Objects with no remaining references become **eligible for garbage collection**.

5.4 Garbage Collection Basics

Java has **automatic memory management**: the JVM allocates objects with `new`, and the **Garbage Collector** frees heap objects that are no longer **reachable** from any live reference. There is no `delete` or `free()`.

```
Employee e = new Employee();
e = null;                        // 1. nullifying the reference

Employee a = new Employee();
a = new Employee();              // 2. reassigning: the first object is unreachable

void work() {
    Employee tmp = new Employee();
}                                // 3. local goes out of scope when the method returns

class Node { Node next; }
Node x = new Node(), y = new Node();
x.next = y; y.next = x;
x = null; y = null;              // 4. "island of isolation": they reference each other,
                                 //    but nothing live reaches them -> eligible
```

REMEMBER

`System.gc()` and `Runtime.getRuntime().gc()` only **request** a collection; the JVM may ignore them. `finalize()` is **deprecated** (Java 9) and was marked for removal in Java 18. Release resources with try-with-resources (Phase 9) instead. GC algorithms are covered in Phase 15.

CORRECTED FROM ORIGINAL NOTES

The original notes call `hashCode()` "a unique integer that identifies objects in memory". It is **not guaranteed unique** (two different objects can share a hash code), and it is **not** a memory address. It is a number used by hash-based collections. `hashCode()`, `equals()` and `toString()` are covered properly in Phase 8.

5.5 Variables in Depth: Instance, Static and Local

Feature	Instance variable	Static (class) variable	Local variable
Declared	In a class, outside methods, no `static`	In a class with `static`	Inside a method, constructor or block (incl. parameters)
Belongs to	Each object	The class (one shared copy)	The method call
Memory	Inside the object on the heap	With the class (allocated at class loading)	Stack frame
Created / destroyed	With the object / when the object is GC'd	When the class loads / when it unloads	On method entry / on method exit
Default value	Yes	Yes	**No**: must be initialised
Access	`obj.var` (directly inside instance methods)	`ClassName.var` (recommended)	Only inside its block
Modifiers allowed	access modifiers, `final`, `transient`, `volatile`	same + `static`	only `final`
Also called	Object / non-static / field	Class variable	Temporary / stack variable

Static data is shared by all objects

```
class Student {
    String name;                     // different for every student
    long phone;
    static String institute = "IIT";   // same for all students -> static
}

Student ankit = new Student();   ankit.name = "Ankit";
Student gautam = new Student();  gautam.name = "Gautam";
Student.institute = "IIT Delhi"; // change once -> every object sees it
System.out.println(ankit.institute + " " + gautam.institute);   // IIT Delhi IIT Delhi
```

```
              +----------------------------+
              | static institute = "IIT..." |   ONE copy, class level
              +-------------+--------------+
                shared by every Student object
          +-------------------+-------------------+
          v                                       v
+------------------+                  +------------------+
| Ankit object     |                  | Gautam object    |
|  name  = "Ankit" |                  |  name = "Gautam" |
|  phone = 0       |                  |  phone = 0       |
+------------------+                  +------------------+
```

> **TIP**
> Use **static** for data common to all objects (company name, interest rate, counters, constants). Use **instance** variables for data specific to each object (name, balance, account number).

Static context cannot see instance members directly

```java
class User {
    int age = 10;                          // instance
    static int count = 0;                  // static

    public static void main(String[] args) {
        // System.out.println(age);        // COMPILE ERROR: non-static from static context
        User u = new User();               // need an object first
        System.out.println(u.age);         // 10
        System.out.println(count);         // static -> accessible directly
    }

    void show() {
        System.out.println(age + count);   // instance methods can use BOTH
    }
}
```

The reason: `main` is static and runs without any object. An instance variable exists only inside an object, so the compiler can't know *which* object's `age` you mean.

5.6 Methods

A **method** is a named block of code that performs a task. Methods make code reusable, testable and readable.

```java
public static int add(int a, int b) throws IllegalArgumentException {
//  |      |    |  |    \___________/  \_________________________________/
//  |      |    |  |  name   parameters        exceptions (optional)
//  |      |    |    return type (void if nothing is returned)
//  |      |    optional modifiers (static, final, abstract, synchronized...)
//  access modifier
    return a + b;    // body / definition / implementation
}
```

Term	Meaning	Example
Method signature	**Name + parameter types** (in order). Used for overloading.	`add(int, int)`
Method declaration / header	Modifiers + return type + signature + throws	`public static int add(int a, int b)`
Method body / definition / implementation	The code inside `{ }`	`{ return a + b; }`
Parameters vs arguments	Variables in the declaration vs actual values passed	`a, b` vs `add(10, 20)`

> **NOTE**
> The return type and the `throws` clause are **not** part of the signature. Two methods that differ only in return type cannot coexist.

Rules

1. Return type and name are mandatory; parameters are optional.
2. A non-`void` method must `return` a value of a compatible type on **every** path.
3. Statements after a `return` in the same block are **unreachable** (compile error). Early `return`s inside `if` blocks are fine.
4. A `void` method may use a bare `return;` to exit early.
5. Methods cannot be nested inside other methods (use lambdas or local classes instead).

> **CORRECTED FROM ORIGINAL NOTES**
> The original notes say "return should be the last statement of a method". More precisely, a method may have several `return` statements (guard clauses are good style). Code placed *after* a `return` in the same block can never run, and the compiler rejects it.

Instance methods vs static methods

	Instance method	Static method
Belongs to	An object	The class
Called as	`obj.method()`	`ClassName.method()`
Can access instance fields / `this`	Yes	**No** (needs an object reference)
Can access static fields	Yes	Yes
Can be overridden	Yes	No: it is *hidden* (Phase 7)
Typical use	Behaviour that depends on object state	Utilities and factories (`Math.max`, `Integer.parseInt`)

```
class Calculator {
    int lastResult;                                 // instance state

    int add(int a, int b) {                         // instance method
        lastResult = a + b;
        return lastResult;
    }
    static int multiply(int a, int b) {             // static utility method
        return a * b;
    }
}

Calculator c = new Calculator();
c.add(2, 3);                    // 5   (needs an object)
Calculator.multiply(2, 3);      // 6   (no object needed)
```

Passing and returning objects

```
class Employee { int id; double salary; }

static void raise(Employee e, double pct) {       // receives a reference copy
    e.salary += e.salary * pct / 100;             // modifies the SAME object
}
static Employee find(int id) {                    // returns an object
    Employee e = new Employee();
    e.id = id;
    e.salary = id == 101 ? 50000 : 30000;
    return e;
}

Employee emp = find(101);
raise(emp, 10);
System.out.println(emp.salary);    // 55000.0
```

5.7 Java Is Always Pass-by-Value

Java passes a **copy** of every argument:

- **Primitives:** the value is copied, so the method can't change the caller's variable.
- **Objects:** the **reference** (address) is copied. The method **can modify the object's fields**, but **cannot make the caller's variable point to a different object**.

```
static void changePrimitive(int x) { x = 100; }

static void changeName(Student s) { s.name = "John"; }   // modifies the shared object

static void replace(Student s) {
    s = new Student();      // only the LOCAL copy of the reference changes
    s.name = "Amit";
}

int n = 5;
changePrimitive(n);
System.out.println(n);              // 5     (unchanged)

Student st = new Student();
st.name = "Rohit";
changeName(st);
System.out.println(st.name);        // John  (object modified)
replace(st);
System.out.println(st.name);        // John  (caller's reference not replaced)
```

```
caller:  st ---------+
                     +---->  [Student name="John"]
method:  s (copy) ---+       after s = new Student():  s ---> [Student name="Amit"]
```

> **INTERVIEW**
> "Is Java pass-by-reference for objects?" **No.** Java passes object *references by value*. Proof: a `swap(a, b)` method that swaps two object parameters has no effect on the caller.

5.8 Variable Arguments (Varargs)

Varargs (Java 5) let a method accept **zero or more** arguments of one type. Inside the method, the parameter is an **array**.

```
static int sum(int... nums) {          // int... behaves like int[]
    int total = 0;
    for (int n : nums) total += n;
    return total;
}
sum();               // 0
sum(5);              // 5
sum(1, 2, 3, 4);     // 10
sum(new int[]{1, 2});// 3  (an array can be passed directly)

static void log(String level, Object... parts) { }   // varargs must be LAST
// static void bad(int... a, String s) { }           // COMPILE ERROR
// static void bad2(int... a, String... b) { }       // COMPILE ERROR: only one varargs
```

Real-world examples include `String.format(String, Object...)`, `Arrays.asList(T...)`, `List.of(E...)` and `System.out.printf`.

5.9 Recursion

A **recursive** method calls itself to solve a smaller instance of the same problem. Every recursive method needs a **base case** that stops the recursion. Each call uses a new **stack frame**, so very deep recursion causes `StackOverflowError`.

```
static long factorial(int n) {          // n! = n * (n-1)!
    if (n <= 1) return 1;               // base case
    return n * factorial(n - 1);        // recursive case
}

static int fib(int n) {                 // 0 1 1 2 3 5 8 13 ...
    if (n < 2) return n;
    return fib(n - 1) + fib(n - 2);     // O(2^n): fine for learning, slow for n > 40
}

static int sumOfDigits(int n) {
    return n == 0 ? 0 : n % 10 + sumOfDigits(n / 10);
}

static boolean isPalindrome(String s, int i, int j) {
    if (i >= j) return true;
    return s.charAt(i) == s.charAt(j) && isPalindrome(s, i + 1, j - 1);
}
```

```
factorial(4)
 = 4 * factorial(3)              stack grows ->  [f(4)] [f(3)] [f(2)] [f(1)]
 = 4 * 3 * factorial(2)
 = 4 * 3 * 2 * factorial(1)      base case returns 1, then frames unwind:
 = 4 * 3 * 2 * 1 = 24            f(2)=2 -> f(3)=6 -> f(4)=24
```

Recursion	Iteration
Elegant for trees, divide-and-conquer, backtracking	Usually faster, constant stack usage
Uses a stack frame per call; risk of `StackOverflowError`	No stack growth
Java does **not** optimise tail calls	Preferred for simple counting loops

5.10 Constructors

A **constructor** is a special block that **initialises a new object**. It runs automatically when `new` is used.

Rules

1. The name must be **exactly the class name**.

2. There is **no return type**, not even `void`. Adding one turns it into an ordinary method!
3. It is called automatically, once per object creation.
4. It can be overloaded, and can be `public`, `protected`, default or `private`.
5. It is **not inherited**, and cannot be `static`, `final`, `abstract` or `synchronized`.
6. If you write **no** constructor, the compiler adds a **default constructor** (no-arg, calls `super()`). If you write **any** constructor, no default is added.

```
class Employee {
    String name;
    double salary;

    Employee() {                              // no-arg constructor
        this("Unknown", 0);                   // this(...) calls another constructor
    }
    Employee(String name) {
        this(name, 25000);
    }
    Employee(String name, double salary) {    // parameterized constructor
        this.name = name;                     // this.name = field, name = parameter
        this.salary = salary;
    }
    Employee(Employee other) {                // copy constructor
        this(other.name, other.salary);
    }
}

Employee e1 = new Employee();                 // Unknown 0.0
Employee e2 = new Employee("Raju");           // Raju 25000.0
Employee e3 = new Employee("Ramu", 40000);    // Ramu 40000.0
Employee e4 = new Employee(e3);               // independent copy of e3
```

```
class Product {
    Product(int id) { }
}
// new Product();        // COMPILE ERROR: no default constructor was generated

class Trap {
    void Trap() { System.out.println("I'm a METHOD, not a constructor!"); }
}
```

> **TIP**
> Run `javap Student` to see the compiler-generated default constructor in the `.class` file.

Constructor vs method

Constructor	Method
Name = class name	Any valid name
No return type	Must declare a return type (or `void`)
Called automatically by `new`	Called explicitly
Initialises object state	Performs behaviour
Not inherited, can't be overridden	Inherited and can be overridden
Compiler supplies a default if none	No default provided

Private constructors

```
public final class MathUtils {            // utility class: static methods only
    private MathUtils() { }               // prevents  new MathUtils()
    public static int square(int x) { return x * x; }
}

public class AppConfig {                   // Singleton: exactly one instance
    private static final AppConfig INSTANCE = new AppConfig();
    private AppConfig() { }                // no outside construction
    public static AppConfig getInstance() { return INSTANCE; }
}
```

5.11 The this Keyword

`this` is a reference to the **current object**: the one whose method or constructor is running.

Use	Example
1. Distinguish a field from a parameter or local with the same name	`this.name = name;`
2. Call another constructor of the same class (must be the **first** statement)	`this("Unknown", 0);`
3. Call another instance method of the current object	`this.validate();`
4. Pass the current object as an argument	`audit.record(this);`
5. Return the current object (method chaining / builders)	`return this;`
6. Refer to the outer instance from an inner class	`Outer.this.field`

```
class Account {
    private double balance;

    Account deposit(double amt) { this.balance += amt; return this; }   // chaining
    Account withdraw(double amt) { this.balance -= amt; return this; }
}
new Account().deposit(500).withdraw(200);    // fluent style
```

PITFALL

Without `this`, `name = name;` inside a constructor assigns the parameter to itself and leaves the field `null`. Also, `this` cannot be used inside a **static** method, because there is no current object.

5.12 The static Keyword

Applies to	Meaning
Variable	One copy per class, shared by all objects
Method	Called on the class; cannot use `this` or instance members directly
Block `static { }`	Runs **once**, when the class is loaded (before `main` for the main class)
Nested class	A nested class that doesn't need an outer instance (Phase 10)
Import	`import static java.lang.Math.*;` then call `sqrt(16)` without `Math.`

```java
class Account {
    private static int counter = 0;              // shared counter
    private static final double MIN_BALANCE;     // static final constant
    private final int accountNo;

    static {                                     // static initializer block
        MIN_BALANCE = 1000;                      // e.g. load from config
        System.out.println("Account class loaded");
    }

    Account() {
        accountNo = ++counter;                   // each object gets the next number
    }
    static int totalAccounts() { return counter; }
}
```

REMEMBER

Static members can be used from anywhere in the class. Instance members need an object. Accessing a static member through an object (`obj.staticVar`) compiles but is misleading; always use `ClassName.staticVar`.

5.13 Initialization Blocks and Execution Order

```java
class Demo {
    static { System.out.println("Static block"); }   // once, at class loading
    { System.out.println("Instance block"); }        // each object, before ctor body
    Demo() { System.out.println("Constructor"); }
}
```

	Static block	Instance block
Runs when	The class is loaded	Each object is created
How often	Once	Every `new`
Runs before	`main()` (for the main class)	The constructor body (after `super()`)
Can access	Static members only	Static and instance members
Used for	Class-level setup (load drivers, config, caches)	Code shared by all constructors

Complete control flow (output verified on JDK 21)

```java
public class ControlFlowDemo {
    static int a = initStatic();                 // static members: in TEXTUAL order
    static { System.out.println("Static Block 1"); }
    static { System.out.println("Static Block 2"); }

    int b = initInstance();                      // instance members: in TEXTUAL order
    { System.out.println("Instance Block 1"); }
    { System.out.println("Instance Block 2"); }

    ControlFlowDemo() { System.out.println("Constructor"); }

    static int initStatic()  { System.out.println("Static Variable Init"); return 10; }
    int initInstance()       { System.out.println("Instance Variable Init"); return 20; }

    public static void main(String[] args) {
        System.out.println("Main Started");
        new ControlFlowDemo();
        System.out.println("--- second object ---");
        new ControlFlowDemo();
    }
}
```

```
Static Variable Init        <- static phase: runs ONCE when the class loads
Static Block 1
Static Block 2
Main Started                <- then main()
Instance Variable Init      <- instance phase: runs for EVERY object
Instance Block 1
Instance Block 2
Constructor
--- second object ---
Instance Variable Init
Instance Block 1
Instance Block 2
Constructor
```

REMEMBER

Order of events:

- **Static flow (once):** static variables and static blocks, top to bottom, then `main()`.
- **Instance flow (each `new`):** `super()` → instance variable initialisers and instance blocks, top to bottom → the rest of the constructor body.

5.14 Predict the Output

```java
class Counter {
    static int s = 0;  int i = 0;
    Counter() { s++; i++; }
}
new Counter(); new Counter(); Counter c = new Counter();
System.out.println(Counter.s + " " + c.i);         // 3 1

static void inc(int[] arr) { arr[0]++; }
int[] data = {5};  inc(data);
System.out.println(data[0]);                       // 6 (array object modified)

static void reassign(int[] arr) { arr = new int[]{99}; }
reassign(data);
System.out.println(data[0]);                       // 6 (caller's reference unchanged)

class A { int x = 10; A() { x = 20; } { x = 30; } }
System.out.println(new A().x);                     // 20 (block runs before ctor body)

class B { void B() { System.out.println("method"); } }
new B();                                           // prints nothing (no constructor call)

static int f(int n) { return n == 0 ? 0 : n + f(n - 1); }
System.out.println(f(4));                          // 10
```

5.15 Interview Questions

Q. What is the difference between a class and an object?

A class is a blueprint that defines fields and methods. An object is a runtime instance of that class occupying heap memory. One class can have many objects.

Q. Where are objects and references stored?

Objects (with their instance fields) always live on the heap. Local reference variables live in the current method's stack frame. Static fields are class-level.

Q. When does an object become eligible for garbage collection?

When no live thread can reach it through any chain of references: after nullifying or reassigning references, when locals go out of scope, or when a group of objects reference only each other (island of isolation).

Q. Is Java pass-by-value or pass-by-reference?

Always pass-by-value. For objects, the value passed is a copy of the reference, so methods can change the object's state but can't reassign the caller's variable.

Q. What is a default constructor? When is it not provided?

A no-argument constructor that the compiler inserts when a class declares no constructors. As soon as you declare any constructor, the compiler doesn't add one.

Q. Can a constructor be private, static, final or have a return type?

It can be private (singletons, utility classes, factories). It cannot be static, final or abstract. If you give it a return type, it becomes a normal method.

Q. What is constructor chaining?

Calling one constructor from another: `this(...)` within the same class, or `super(...)` to the parent. The call must be the first statement, so you can't use both in one constructor.

Q. Why can't a static method access instance variables directly?

A static method runs without any object, so there is no `this` to say whose instance variable is meant. It must first obtain an object reference.

Q. What is the order of execution of static blocks, instance blocks and constructors?

Static variables and blocks run once at class loading, in textual order. For each object: the parent constructor, then instance initialisers and blocks in textual order, then the rest of the constructor body.

Q. What are varargs and their rules?

`type... name` accepts zero or more arguments as an array. It must be the last parameter, and a method can have only one.

Q. What causes `StackOverflowError` in recursion?

A missing or unreachable base case (or input that is too deep): each call adds a stack frame until the thread's stack is exhausted.

5.16 Quick Revision

- Class = blueprint; object = instance on the **heap**; reference = address held in a variable.
- `new` allocates memory → fields get defaults → constructor runs → reference returned.
- An object is GC-eligible when unreachable; `System.gc()` is only a request; `finalize()` is deprecated.
- Instance variables: per object, defaults. Static: per class, shared. Local: per call, **no defaults**.
- Static code can't touch instance members without an object; there is no `this` in static methods.
- Signature = name + parameter types (not the return type).
- **Always pass-by-value**: object methods can mutate the object but can't rebind the caller's variable.
- Varargs: `type...`, last parameter only, one per method, seen as an array.
- Recursion needs a base case; each call uses a stack frame.
- Constructor: class name, no return type, not inherited; default constructor only when none is written.
- `this(...)` / `super(...)` must be the first statement in a constructor.
- Order: static vars/blocks (once) → `main` → per object: `super()` → instance vars/blocks → constructor body.